Chapter 7. Hardware and Operating Systems

Why should Java developers care about hardware?

For many years the computer industry has been driven by Moore's law, a hypothesis made by Intel founder Gordon Moore about long-term trends in processor capability. The law (really an observation or extrapolation) can be framed in a variety of ways, but one of the most usual is:

> *The number of transistors on a mass-produced chip roughly doubles every 18 months.*
>
> —Moore's law (informally)

This phenomenon represents an exponential increase in computer power over time. It was originally cited in 1965, so represents an incredible long-term trend, unparalleled in the history of computation. The effects of Moore's law have been transformative in many (if not most) areas of the modern world. The death of Moore's law has been repeatedly proclaimed for decades now. However, there are very good reasons to suppose that, for all practical purposes, this incredible progress in chip technology has (finally) come to an end:

> *Transistors can only get so small and, eventually, the more permanent laws of physics get in the way. Already transistors can be measured on an atomic scale, with the smallest ones commercially available only 3 nanometers wide, barely wider than a strand of human DNA (2.5nm). While there's still room to make them smaller (in 2021, IBM announced the successful creation of 2-nanometer chips), such progress has become prohibitively expensive and slow, putting reliable gains into question. And there's still the physical limitation in that wires can't be thinner than atoms, at least not with our current understanding of material physics.*
>
> — Audrey Woods, "The Death of Moore's Law: What it means and what might fill the gap going forward"

Hardware has become increasingly complex to make good use of the "transistor budget" available in modern computers. The software platforms that run on that hardware have also increased in complexity to exploit the new capabilities. More horsepower is available, but engineers

have to do more work in software to harness it, so the performance delivered is diminished by this overhead.

Software applications now pervade (nearly) every aspect of global society. And that software is becoming increasingly complex, as application developers take advantage of the performance available.

Or, to put it another way:

> *Software is eating the world.*
>
> —Marc Andreessen

As we will see, Java has been a beneficiary of the increasing amount of computer power. The design of the language and runtime has been well suited (or lucky) to use this trend in processor capability, as we have discussed in previous chapters. However, the truly performance-conscious Java programmer needs to understand the principles and technology that underpin the platform to best use the available resources.

This is particularly pertinent because, as we will see in later chapters, the development of cloud native applications has somewhat changed the performance landscape. But before turning to those subjects, let's take a quick look at modern hardware and operating systems, as an understanding of those subjects will help with everything that follows.

Introduction to Modern Hardware

Many university courses on hardware architectures still teach a simple-to-understand, classical view of hardware. This abstracted logical view of hardware focuses on a simple register-based machine, with arithmetic, logic, and load and store operations. Since the 1990s, the world of the application developer has, to a large extent, revolved around the Intel x86/x64 architecture (and more recently the rise of the ARM chip).

However, this is an area of technology that has undergone radical change. The simple mental model of a processor's operation is now incorrect, and intuitive reasoning based on it is liable to lead to utterly wrong conclusions.

To help address this, in this chapter, we will discuss several of these advances in CPU technology. We will start with the behavior of memory, as this is by far the most important to a modern Java developer.

Memory

As Moore's law advanced, the exponentially increasing number of transistors was initially used for faster and faster clock speed. The reasons for this are obvious: faster clock speed means more instructions completed per second. Accordingly, the speed of processors has advanced hugely, and the 2+ GHz processors that we have today are hundreds of times faster than the original 4.77 MHz chips found in the first IBM PC.

However, the increasing clock speeds uncovered another problem—faster chips require a faster stream of data to act upon. As Figure 7-1 shows,[1] over time, main memory could not keep up with the demands of the processor core for fresh data.

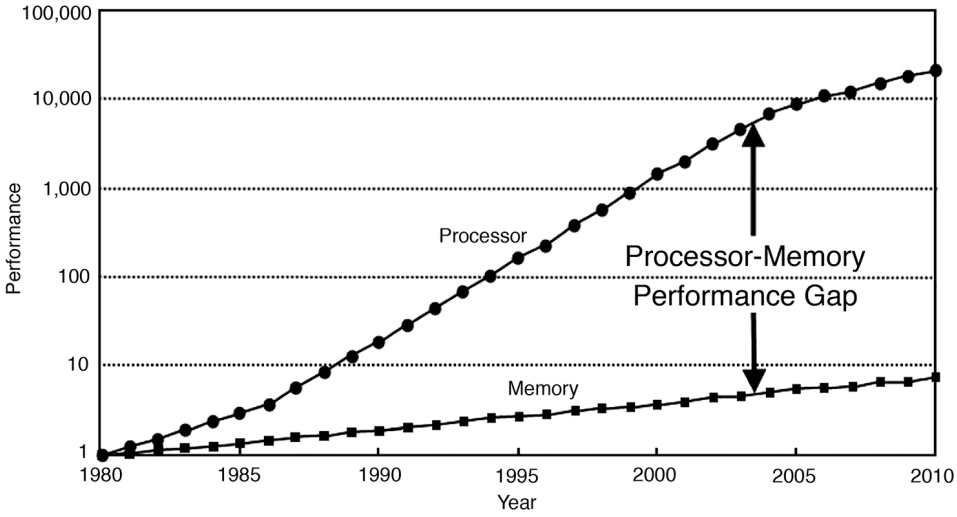

Figure 7-1. Gap between processor and memory performance capabilities (Hennessy and Patterson, 2011)

This results in a problem: if the CPU is waiting for data, then faster cycles don't help, as the CPU will just have to idle until the required data arrives.

Memory Caches

To solve this problem, CPU caches were introduced. These are memory areas on the CPU that are slower than CPU registers but faster than main memory. The idea is for the CPU to fill the cache with copies of often-accessed memory locations rather than constantly having to re-address main memory.

Modern CPUs have several layers of cache, with the most-often-accessed caches being located close to the processing core. The cache closest to the CPU is usually called *L1* (for "level 1 cache"), with the next being referred to as *L2*, and so on. Different processor architectures have a varying number and configuration of caches, but a common choice is for each execution core to have a dedicated, private L1 and L2 cache, and an L3 cache

that is shared across some or all of the cores. The effect of these caches in speeding up access times is shown in Figure 7-2.[2]

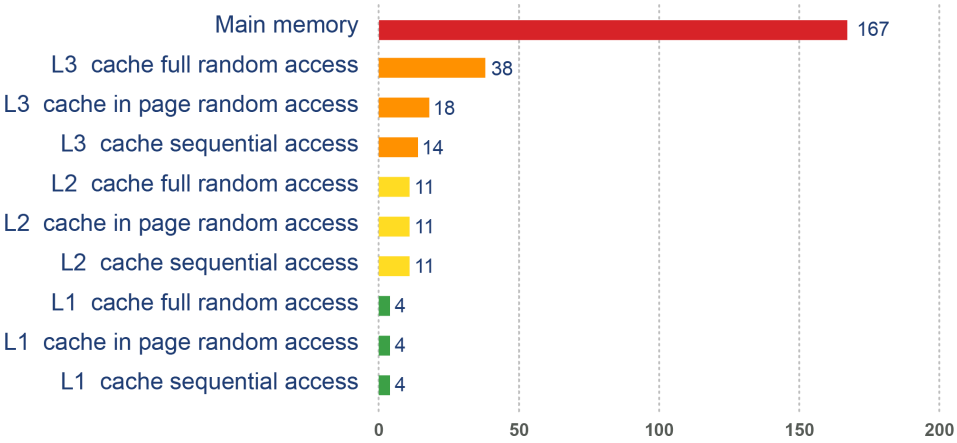

Figure 7-2. Access times for various types of memory

This approach to cache architecture improves access times and helps keep the core fully stocked with data to operate on. Due to the clock speed versus access time gap, more transistor budget is devoted to caches on a modern CPU.

The resulting design can be seen in Figure 7-3. This shows the L1 and L2 caches (private to each CPU core) and a shared L3 cache that is common to all cores on the CPU. Main memory is accessed over the Northbridge component, and it is traversing this bus that causes the large drop-off in access time to main memory.

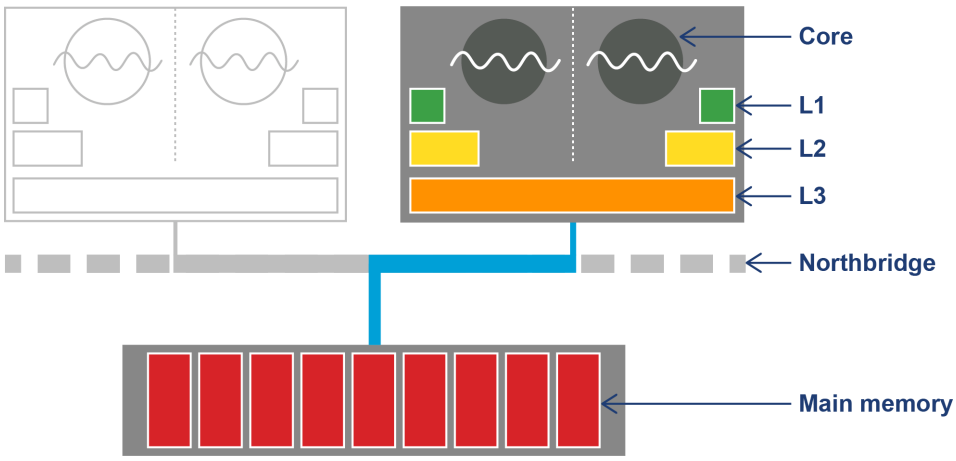

Figure 7-3. Overall CPU and memory architecture

Although the addition of a caching architecture hugely improves processor throughput, it introduces a new set of problems. These problems include determining how memory is fetched into and written back from the cache. The solutions to this problem are usually referred to as *cache consistency protocols*.

NOTE

There are other problems that crop up when this type of caching is applied in a parallel processing environment, as we will see later in this book.

At the lowest level, a protocol called MESI (and its variants) is commonly found on a wide range of processors. It defines four states for any line in a cache. Each line (usually 64 bytes) is:

- Modified (but not yet flushed to main memory)
- Exclusive (present only in this cache, but does match main memory)
- Shared (may also be present in other caches; matches main memory)
- Invalid (may not be used; will be dropped as soon as practical)

The idea of the protocol is that multiple processors can simultaneously be in the Shared state. However, if a processor transitions to any of the other valid states (Exclusive or Modified), then this will force all the other processors into the Invalid state. This is shown in Table 7-1, where `Y` indicates a permitted transition and – represents a transition that is not permitted.

Table 7-1. MESI allowable states between processors

	M	**E**	**S**	**I**
M	-	-	-	Y
E	-	-	-	Y
S	-	-	Y	Y
I	Y	Y	Y	Y

The protocol works by broadcasting the intention of a processor to change state. An electrical signal is sent across the shared memory bus, and the other processors are made aware. The full logic for the state transitions is shown in Figure 7-4.

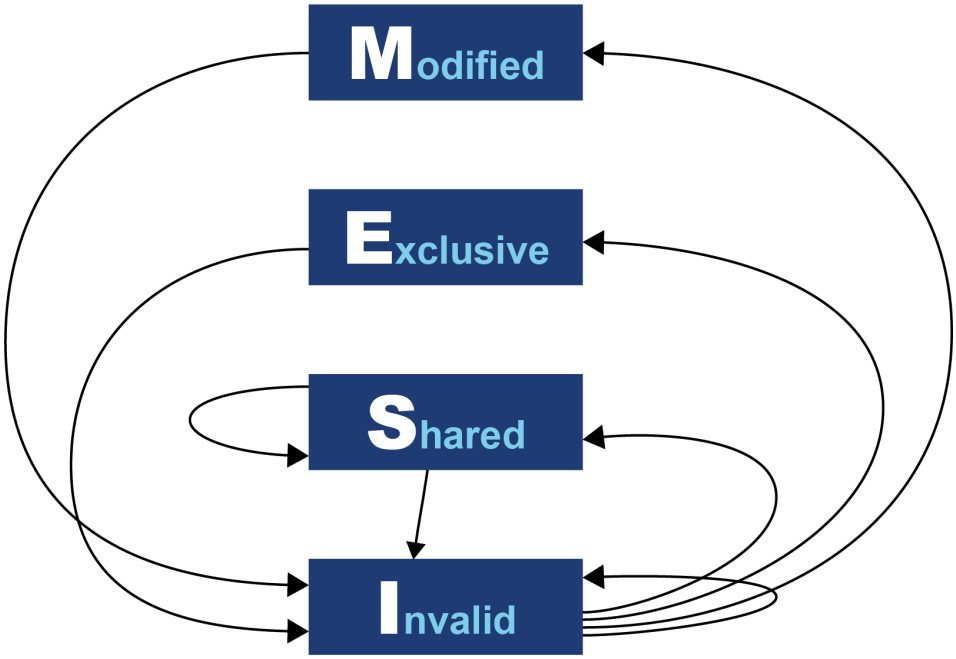

Figure 7-4. MESI state transition diagram

Originally, processors wrote every cache operation directly into main memory. This was called *write-through* behavior, but it was and is very inefficient, and it required a large amount of bandwidth to memory. More recent processors also implement *write-back* behavior, where traffic back to main memory is significantly reduced by processors writing only modified (dirty) cache blocks to memory when the cache blocks are replaced.

The overall effect of caching technology is to greatly increase the speed at which data can be written to, or read from, memory. This is expressed in terms of the bandwidth to memory. The *burst rate*, or theoretical maximum, is based on several factors:

- Clock frequency of memory
- Width of the memory bus (usually 64 bits)
- Number of interfaces (usually two in modern machines)

This is multiplied by two in the case of DDR RAM (DDR stands for "double data rate" as it communicates on both edges of a clock signal).

Applying the formula to 2024 commodity hardware gives a theoretical maximum write speed of 17+ GB/s per core, with overall system bandwidth of 70–90 GB/s.[3] In practice, of course, this could be limited by many other factors in the system. As it stands, this gives a modestly useful value to allow us to see how close the hardware and software can get.

Let's write some simple code to exercise the cache hardware, as seen in Example 7-1.

Example 7-1. Caching example

```
public class Caching {
```

```
    private final int ARR_SIZE = 2 * 1024 * 1024;
    private final int[] testData = new int[ARR_SIZE];

    private void run() {
        System.err.println("Start: "+ System.currentTimeMillis());
        for (int i = 0; i < 15_000; i++) {
            touchEveryLine();
            touchEveryItem();
        }
        System.err.println("Warmup finished: "+ System.currentTimeMillis())
        System.err.println("Item     Line");
        for (int i = 0; i < 100; i++) {
            long t0 = System.nanoTime();
            touchEveryLine();
            long t1 = System.nanoTime();
            touchEveryItem();
            long t2 = System.nanoTime();
            long elItem = t2 - t1;
            long elLine = t1 - t0;
            double diff = elItem - elLine;
            System.err.println(elItem + " " + elLine +" "+  (100 * diff / el
        }
    }

    private void touchEveryItem() {
        for (int i = 0; i < testData.length; i++)
            testData[i]++;
    }

    private void touchEveryLine() {
        for (int i = 0; i < testData.length; i += 16)
            testData[i]++;
    }

    public static void main(String[] args) {
        Caching c = new Caching();
        c.run();
    }
}
```

Intuitively, `touchEveryItem()` does 16 times as much work as `touchEveryLine()`, as 16 times as many data items must be updated. However, the point of this simple example is to show how badly intuition can lead us astray when dealing with JVM performance. Let's look at some sample output from the `Caching` class, as shown in Figure 7-5.

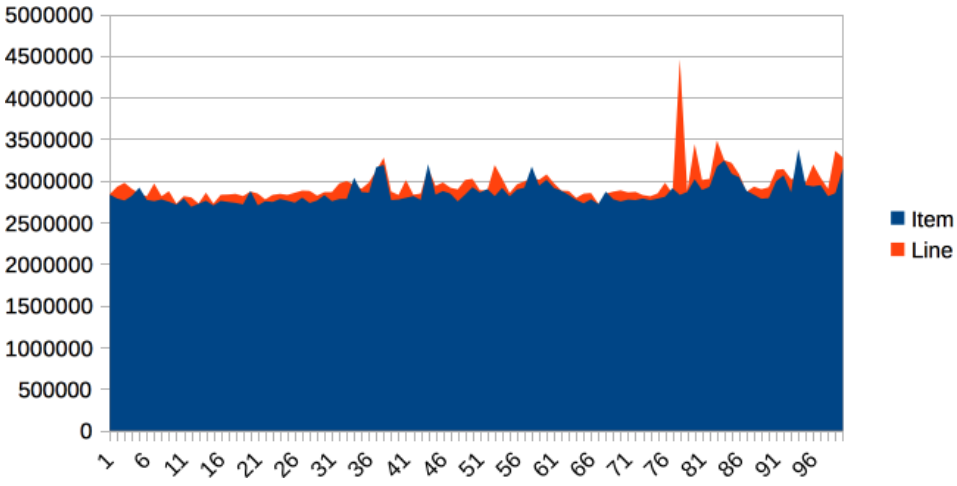

Figure 7-5. Time (ns) elapsed for caching example

The graph shows 100 runs of each function and is intended to show several different effects. First, notice that the results for both functions are remarkably similar in terms of time taken, so the intuitive expectation of "16 times as much work" is clearly false.

Instead, the dominant effect of this code is to exercise the memory bus by transferring the contents of the array from main memory into the cache to be operated on by `touchEveryItem()` and `touchEveryLine()`.

In terms of the statistics of the numbers, although the results are reasonably consistent, there are individual outliers that are 30%–35% different from the median value.

Overall, we can see that each iteration of the simple memory function takes around 3 milliseconds (2.86 ms on average) to traverse a 100 MB chunk of memory, giving an effective memory bandwidth of just under 3.5 GB/s. This is less than the theoretical maximum, but is still a reasonable number. Designing for theoretical limits can be a recipe for disappointment (or disaster). Empirical numbers are useful for baselines and planning purposes. Results that are significantly different—i.e., orders of magnitude—from theoretical limits might be considered unreasonable and warrant further exploration.

NOTE

Modern CPUs have a hardware prefetcher that can detect predictable patterns in data access (usually just a regular "stride" through the data). In this example, we're taking advantage of that fact to get closer to a realistic maximum for memory access bandwidth.

One of the key themes in Java performance is the sensitivity of applications to object allocation rates. We will return to this point several times, but this simple example gives us a basic yardstick for the upper limit of allocation rates.

Modern Processor Features

Hardware engineers sometimes refer to the new features that have become possible as a result of Moore's law as "spending the transistor budget." Memory caches are the most obvious use of the growing number of transistors, but other techniques have also appeared over the years.

Translation Lookaside Buffer

One very important use of hardware caching is the translation lookaside buffer (TLB), a hardware cache for the *page tables* that map virtual memory locations (which are the ones seen by application code) to hardware locations. This greatly speeds up a very frequent operation—access to the physical address underlying a virtual address.

NOTE

We have already met the TLAB feature of HotSpot's GC in Chapter 4, and some texts refer to a TLAB as a TLB, which can be confusing as the two concepts are not related. Always check which feature is being discussed when you see TLB mentioned.

Without the TLB, all virtual address lookups would take 16 cycles, even if the page table was held in the L1 cache. The resulting performance would be unacceptable, so the TLB is basically essential for all modern chips.

Branch Prediction and Speculative Execution

One of the advanced processor tricks that appears on modern processors is branch prediction. This is used to prevent the processor from having to wait to evaluate a value needed for a conditional branch. Modern processors have multistage instruction pipelines. This means that the execution of a single CPU cycle is broken down into a number of separate stages. There can be several instructions in flight (at different stages of execution) at once.

In this model a conditional branch is problematic, because until the condition is evaluated, it isn't known what the next instruction after the branch will be. This can cause the processor to stall for a number of cycles (in practice, up to 20), as it effectively empties the multistage pipeline behind the branch.

NOTE

Speculative execution was, famously, the cause of major security problems (including Meltdown and Spectre) that were discovered to affect very large numbers of CPUs in early 2018.

To avoid this, the processor can dedicate transistors to building up a heuristic to decide which branch is more likely to be taken. Using this guess, the CPU fills the pipeline based on a gamble. If it works, then the CPU carries on as though nothing had happened. If it's wrong, then the partially executed instructions are dumped, and the CPU has to pay the penalty of emptying the pipeline.

Hardware Memory Models

The core question about memory that must be answered in a multicore system is "How can multiple different CPUs access the same memory location consistently?"

The answer to this question is highly hardware-dependent, but in general, `javac`, the JIT compiler, and the CPU are all allowed to make changes to the order in which code executes. This is subject to the provision that any changes don't affect the outcome as observed by the current thread.

For example, suppose we have a piece of code like this:

```
myInt = otherInt;
intChanged = true;
```

There is no code between the two assignments, so the executing thread doesn't need to care about what order they happen in, and thus the environment is at liberty to change the order of instructions.

However, this could mean that in another thread that has visibility of these data items, the order could change, and the value of `myInt` read by the other thread could be the old value, despite `intChanged` being seen to be `true`.

This type of reordering (stores moved after stores) is not possible on x86 chips, but as Table 7-2 shows, there are other architectures where it can, and does, happen.

Table 7-2. Hardware memory support

	ARMv7	POWER	SPARC	x86	AMD64	zSeries
Loads moved after loads	Y	Y	-	-	-	-
Loads moved after stores	Y	Y	-	-	-	-
Stores moved after stores	Y	Y	-	-	-	-
Stores moved after loads	Y	Y	Y	Y	Y	Y
Atomic moved with loads	Y	Y	-	-	-	-
Atomic moved with stores	Y	Y	-	-	-	-
Incoherent instructions	Y	Y	Y	Y	-	Y

In the Java environment, the Java Memory Model (JMM) is explicitly designed to be a weak model to take into account the differences in consistency of memory access between processor types. Correct use of locks and volatile access is a major part of ensuring that multithreaded code works properly. This is a very important topic that we will return to in Chapter 13.

A trend in recent years has been for software developers to seek greater understanding of the workings of hardware to derive better performance. The term *mechanical sympathy* has been coined to describe this approach, especially as applied to the low-latency and high-performance spaces.

> *The name "mechanical sympathy" comes from the great racing driver Jackie Stewart, who was a three times world Formula 1 champion. He believed the best drivers had enough understanding of how a machine worked so they could work in harmony with it.*
>
> —Martin Thompson

It can be seen in recent research into lock-free algorithms and data structures, which we will meet in Chapter 13.

Operating Systems

The point of an operating system is to control access to resources that must be shared between multiple executing processes. All resources are finite, and all processes are greedy, so the need for a central system to arbitrate and meter access is essential. Among these scarce resources, the two most important are usually memory and CPU time.

Virtual addressing via the memory management unit (MMU) and its page tables is the key feature that enables access control of memory and prevents one process from damaging the memory areas owned by another.

The TLBs that we met earlier in the chapter are a hardware feature that improves lookup times to physical memory. The use of the buffers improves performance for software's access time to memory. However, the MMU is usually too low level for developers to measure.

Instead, let's take a closer look at the OS process scheduler, as this controls access to the CPU and is a far more user-visible piece of the operating system kernel.

The Scheduler

The job of the process scheduler is to manage access to the CPU cores (and to respond to interrupts). On a modern system there are almost always more platform threads that can run simultaneously, so this CPU contention requires an access control mechanism to resolve it.

NOTE

In this section, we are explicitly talking about the OS-level platform threads. Java 21+ virtual threads do not follow this model—instead, it would be the *carrier threads* that virtual threads are multiplexed onto, which obey this model. See Chapter 13 for more details.

This access control uses a queue known as the *run queue* as a waiting area for the *platform threads* that are eligible to run but must wait their turn for the CPU. The overall lifecycle of a platform thread is shown in Figure 7-6.

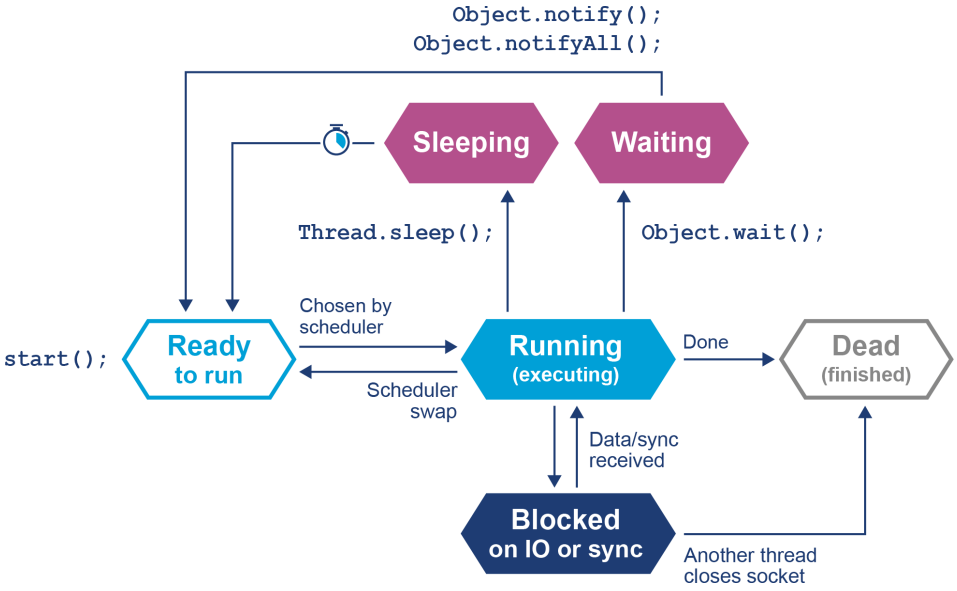

Figure 7-6. Thread lifecycle

In this relatively simple view, the OS scheduler moves platform threads on and off the single core in the system. At the end of the time quantum (often 10 ms or 100 ms in older operating systems), the scheduler moves the thread to the back of the run queue to wait until it reaches the front of the queue and is eligible to run again.

If a thread wants to voluntarily give up its time quantum, it can do so either for a fixed amount of time (via `sleep()`) or until a condition is met (using `wait()`). Finally, a thread can also block on I/O or a software lock.

When you are meeting this model for the first time, it may help to think about a machine that has only a single execution core. Real hardware is, of course, more complex, and virtually any modern machine will have multiple cores, which allows for true simultaneous execution of multiple execution paths. This means that reasoning about execution in a true multiprocessing environment is very complex and counterintuitive.

An often-overlooked feature of operating systems is that, by their nature, they introduce periods of time when code of interest is not running on the CPU. A process that has completed its time quantum will not get back on the CPU until it arrives at the front of the run queue again. Combined with the fact that CPU is a scarce resource, this means that code is waiting more often than it is running.

This means that the statistics we want to generate from processes that we actually want to observe are affected by the behavior of other processes on the systems. This "jitter" and the overhead of scheduling is a primary cause of noise in observed results. We discussed the statistical properties and handling of real results in Chapter 2 and observed this in `Caching.java`.

One of the easiest ways to see the action and behavior of a scheduler is to try to observe the overhead imposed by the OS to achieve scheduling. The

following code executes 1,000 separate 1 ms sleeps. Each of these sleeps will involve the thread being sent to the back of the run queue and having to wait for a new time quantum. So, the total elapsed time of the code gives us some idea of the overhead of scheduling for a typical process:

```
long start = System.currentTimeMillis();
for (int i = 0; i < 1_000; i++) {
    Thread.sleep(1);
}
long end = System.currentTimeMillis();
System.out.println("Millis elapsed: " + (end - start) / 1000.0);
```

Running this code will cause wildly divergent results, depending on the operating system. Most Unixes will report roughly 10%–20% overhead. Earlier versions of Windows had notoriously bad schedulers, with some versions of Windows XP reporting up to 180% overhead for scheduling (so that 1,000 sleeps of 1 ms would take 2.8 s).

There are even reports that some proprietary OS vendors have inserted code into their releases to detect benchmarking runs and cheat the metrics.

Now that we understand the scheduling quantum and the impact that has on process thread execution, let's delve a little deeper into how the JVM typically calls into the operating system when required.

The JVM and the Operating System

The JVM provides a portable execution environment that is independent of the operating system by providing a common interface to Java code. However, for some fundamental services, such as thread scheduling (or even something as mundane as getting the time from the system clock), the underlying operating system must be accessed.

This capability is provided by native methods, which are denoted by the keyword `native`. They are written in C but are accessible as ordinary Java methods. This interface is referred to as the Java Native Interface (JNI). For example, `java.lang.Object` declares multiple nonprivate native methods; all these methods deal with relatively low-level platform concerns.

Let's look at a more straightforward and familiar example: getting the system time.

Consider the `os::javaTimeMillis()` function. This is the (system-specific) code responsible for implementing the Java `System.currentTimeMillis()` static method. The code that does the

actual work is implemented in C++ but is accessed from Java via a "bridge" of C code. Let's look at how this code is actually called in HotSpot.

As you can see in Figure 7-7, the native `System.currentTimeMillis()` method is mapped to the JVM entry point method `JVM_CurrentTimeMillis()`. This mapping is achieved via the JNI `Java_java_lang_System_registerNatives()` mechanism contained in `java/lang/System.c`.

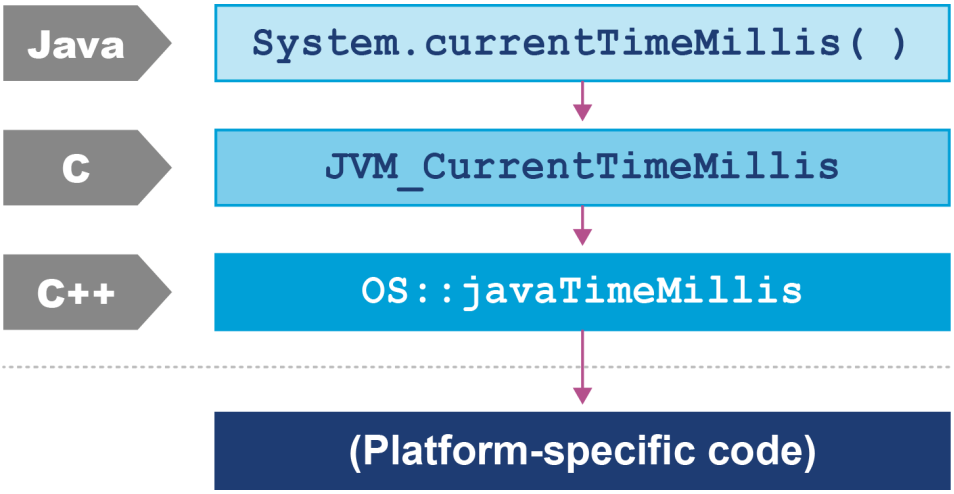

Figure 7-7. The HotSpot calling stack

`JVM_CurrentTimeMillis()` is a call to the VM entry point method. This presents as a C function but is really a C++ function exported with a C calling convention. The call boils down to the call `os::javaTimeMillis()` wrapped in a couple of OpenJDK macros.

This method is defined in the `os` namespace and is operating system dependent. Definitions for this method are provided by the OS-specific subdirectories of source code within OpenJDK. This provides a simple demonstration of how the platform-independent parts of Java can call into services that are provided by the underlying operating system and hardware.

Let's look at a related subject, which is what happens when the scheduler needs to change which threads are currently executing.

Context Switches

A *context switch* is the process by which the OS scheduler removes a currently running platform thread and replaces it with another. There are several different types of context switch, but broadly speaking, they all involve swapping the executing instructions and the stack state of the thread.

A context switch can be a costly operation, whether between user threads or from user mode into kernel mode (sometimes called a *mode switch*).

The latter case is particularly important, because a user thread may need to swap into kernel mode to perform some function partway through its time slice. However, this switch will force instruction and other caches to be emptied, as the memory areas accessed by the user space code will not normally have anything in common with the kernel.

A context switch into kernel mode will invalidate the TLBs and potentially other caches. When the call returns, these caches will have to be refilled, so the effect of a kernel mode switch persists even after control has returned to user space. This causes the true cost of a system call to be masked.[4]

For example, Figure 7-8 highlights the cost of an inter-process communication (IPC) call—it's made in user mode but requires a switch to kernel mode. After the switch happens (represented as a "syscall exception," but note that this is not a Java exception, but an interrupt), then performance drops and only slowly recovers as the cache refills.

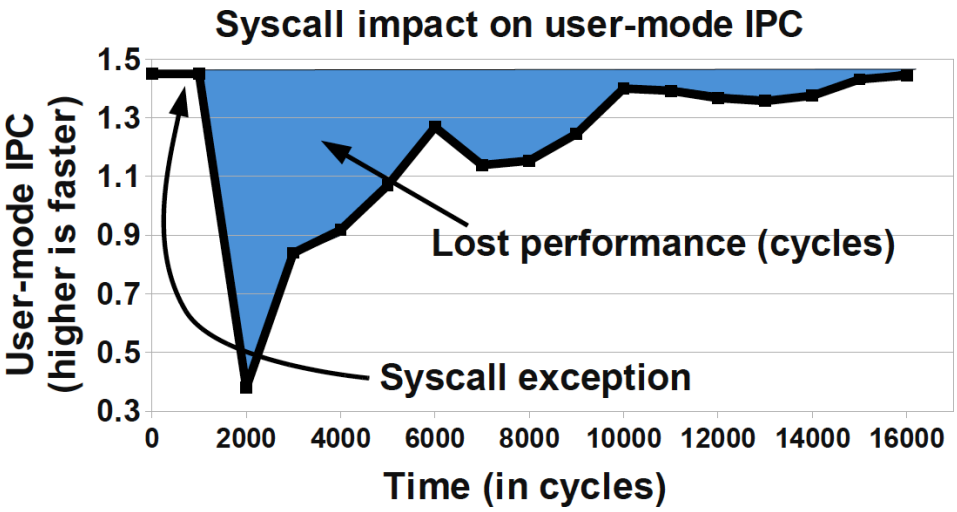

Figure 7-8. Impact of a system call (Soares and Stumm, 2010)

To mitigate this when possible, Linux provides a mechanism known as the *virtual dynamic shared object* (vDSO). This is a memory area in user space that is used to speed up syscalls that do not really require kernel privileges. It achieves this speed increase by not actually performing a context switch into kernel mode. Let's look at an example to see how this works with a real syscall.

For example, a very common Unix system call is `gettimeofday()`. This returns the "wallclock time" as understood by the operating system. Behind the scenes, it is actually just reading a kernel data structure to obtain the system clock time. As this is side-effect free, it does not need privileged access.

If we can use the vDSO to arrange for this data structure to be mapped into the address space of the user process, then there's no need to per-

form the switch to kernel mode. As a result, the refill penalty shown in Figure 7-7 does not have to be paid.

Given how often most Java applications need to access timing data, this is a welcome performance boost. The vDSO mechanism generalizes this example slightly and can be a useful technique, even if it is available only on Linux.

A Simple System Model

In this section, we cover a simple model for describing basic sources of possible performance problems. The model is expressed in terms of operating system observables of fundamental subsystems and can be directly related back to the outputs of standard Unix command-line tools.

NOTE

This may seem too low level or antiquated by the standards of modern cloud applications, but the point is to establish a foundational model that we can then use with the sources of data (e.g., observability data) that we will actually use to diagnose problems.

The model is based on a simple conception of a Java application running on a Unix or Unix-like operating system. Figure 7-9 shows the basic components of the model, which consist of:

- The hardware and operating system the application runs on
- The JVM (or container) the application runs in
- The application code itself
- Any external systems the application calls
- The incoming request traffic that is arriving at the application

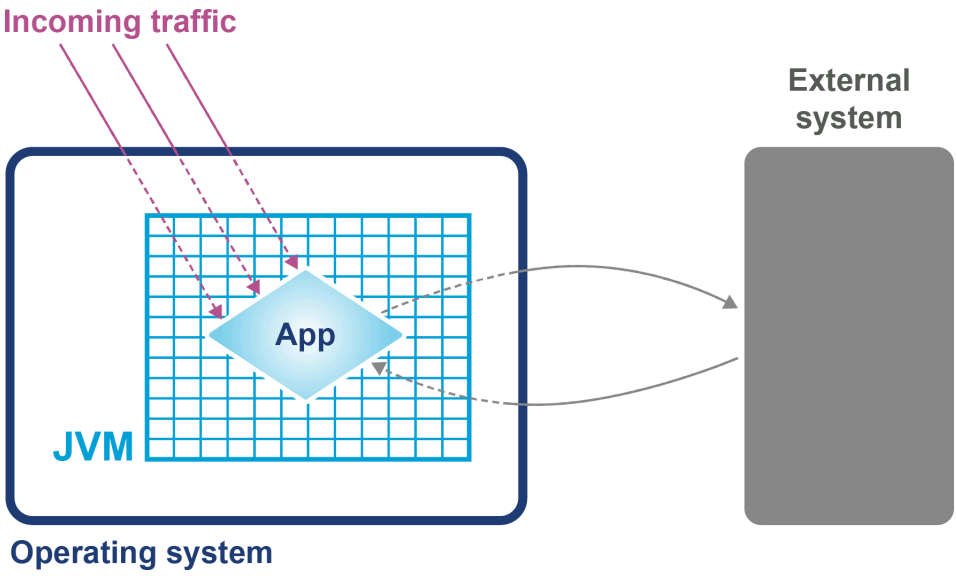

Figure 7-9. Simple system model

Any of these aspects of a system can be responsible for a performance bottleneck. Some simple diagnostic techniques can be used to narrow down or isolate particular parts of the system as potential culprits for performance issues.

In fact, one definition for a well-performing application is that efficient use is being made of system resources. This includes CPU usage, memory, and network or I/O bandwidth.

TIP

If an application is causing one or more resource limits to be hit, then the result will be a performance problem.

The first step in any performance diagnosis is to recognize which resource limit is being hit. We cannot tune performance without dealing with the resource shortage—either by increasing the available resources or the efficiency of use.

It is also worth noting that the operating system itself should not normally be a major contributing factor to system utilization—the role of an operating system is to manage resources on behalf of user processes, not to consume them itself.

The only real exception to this rule is when resources are so scarce that the OS is having difficulty allocating anywhere near enough to satisfy user requirements. For most modern server-class hardware, the only time this should occur is when I/O (or occasionally memory) requirements greatly exceed capability.

Utilizing the CPU

A key metric for application performance is CPU utilization. CPU cycles are quite often the most critical resource needed by an application, so efficient use of them is essential for good performance. CPU-bound applications should be aiming for as close to 100% usage as possible during periods of high load, although this is difficult to achieve in practice due to other dependencies of your application. Looking at your process at a high level helps to reveal constraints on your application.

TIP

When you are analyzing application performance, the system must be under enough load to exercise it. The behavior of an idle application is usually meaningless for performance work.

Three basic tools that every performance engineer should be aware of are `vmstat`, `ifstat`, and `iostat`.

`vmstat`

Reports statistics on virtual memory, including information about sizing, IO, and accessing

`ifstat`

Provides statistics on the network interface and would be used as a starting point to debug network-level process interaction

`iostat`

Monitors input/output on devices and would be used to identify any device interactions causing an issue

On Linux and other Unixes, these command-line tools provide immediate and often very useful insight into the current state of the virtual memory and I/O subsystems, respectively.

The tools only provide numbers at the level of the entire host, but this is frequently enough to point the way to more detailed diagnostic approaches. Let's look at how to use `vmstat` as an example:

```
$ vmstat 1
 r  b swpd  free    buff  cache   si   so  bi  bo   in   cs us sy  id wa st
 2  0    0 759860 248412 2572248    0    0   0  80   63  127  8  0  92  0  0
 2  0    0 759002 248412 2572248    0    0   0   0   55  103 12  0  88  0  0
 1  0    0 758854 248412 2572248    0    0   0  80   57  116  5  1  94  0  0
 3  0    0 758604 248412 2572248    0    0   0  14   65  142 10  0  90  0  0
 2  0    0 758932 248412 2572248    0    0   0  96   52  100  8  0  92  0  0
 2  0    0 759860 248412 2572248    0    0   0   0   60  112  3  0  97  0  0
```

The parameter `1` following `vmstat` indicates that we want `vmstat` to provide ongoing output at a frequency of 1 sample per second (until interrupted via Ctrl-C) rather than a single snapshot. New output lines are printed every second, which enables a performance engineer to leave this output running (or capture it into a log) while an initial performance test is performed.

The output of `vmstat` is relatively easy to understand and contains a large amount of useful information, divided into sections:

1. The first two columns show the number of runnable (`r`) and blocked (`b`) processes.
2. In the memory section, the amount of swapped and free memory is shown, followed by the memory used as buffer and as cache.

3. The swap section shows the memory swapped in from and out to disk (`si` and `so`). Modern server-class machines should not normally experience very much swap activity.
4. The block in and block out counts (`bi` and `bo`) show the number of 512-byte blocks that have been received from and sent to a block (I/O) device.
5. In the system section, the number of interrupts (`in`) and the number of context switches per second (`cs`) are displayed.
6. The CPU section contains a number of directly relevant metrics, expressed as percentages of CPU time. In order, they are user time (`us`), kernel time (`sy`, for "system time"), idle time (`id`), waiting time (`wa`), and "stolen time" (`st`, for virtual machines).

Over the course of the remainder of this book, we will meet many other, more sophisticated tools. However, it is important not to neglect the basic tools at our disposal. Complex tools may have behaviors that can mislead us, whereas the simple tools that operate close to processes and the operating system can convey clear, uncluttered views of how our systems are actually behaving.

Let's consider an example. In "The JVM and the Operating System", we discussed the impact of a context switch, and we saw the potential impact of a full context switch to kernel space in Figure 7-7. However, whether between user threads or into kernel space, context switches introduce unavoidable wastage of CPU resources.

A well-tuned program should be making maximum possible use of its resources, especially CPU. For workloads that are primarily dependent on computation ("CPU-bound" problems), the aim is to achieve close to 100% utilization of CPU for userland work.

To put it another way, if we observe that the CPU utilization is not approaching 100% user time, then the next obvious question is, "Why not?" What is causing the program to fail to achieve that? Are involuntary context switches caused by locks the problem? Is it due to blocking caused by I/O contention?

The `vmstat` tool can, on most operating systems (especially Linux), show the number of context switches occurring, so a `vmstat 1` run allows the analyst to see the real-time effect of context switching. A process that is failing to achieve 100% userland CPU usage and is also displaying a high context switch rate is likely to be blocked on I/O, experiencing thread lock contention, or is written in such a way that it is causing unnecessary context switches.

However, `vmstat` output is not enough to fully disambiguate these cases on its own— `vmstat` can only help indicate I/O problems, as it provides only a crude view of I/O operations. More detailed diagnosis would be

possible with tools like JMC (on the desktop) or Java Flight Recorder (or commercial profiling tools). See Chapters 10, 11, and 12 for more details.

Garbage Collection

As we saw in Chapter 4, in the HotSpot JVM (by far the most commonly used JVM), memory is allocated at startup and managed from within user space. That means that system calls (such as `sbrk()`) are not needed to allocate memory. In turn, this means that kernel-switching activity for garbage collection is quite minimal.

Thus, if a system is exhibiting high levels of system CPU usage, then it is definitely not spending a significant amount of its time in GC, as GC activity burns user space CPU cycles and does not impact kernel space utilization.

On the other hand, if a JVM process is using 100% (or close to that) of CPU in user space, then garbage collection could be the culprit. When analyzing a performance problem, if simple tools (such as `vmstat`) show consistent 100% CPU usage but with almost all cycles being consumed by user space, then we should ask, "Is it the JVM or user code that is responsible for this utilization?"

In many cases, high user space utilization by the JVM is caused by the GC subsystem, so a useful rule is to check the GC log and see how often new entries are being added to it.

Garbage collection logging in the JVM is incredibly cheap, to the point that even the most accurate measurements of the overall cost cannot reliably distinguish it from random background noise. GC logging is also incredibly useful as a source of data for analytics. It is therefore imperative that GC logs be enabled for all JVM processes, especially in production.

We would encourage the reader to consult with their operations staff and confirm whether GC logging is on in production. Observability tools, such as those we will discuss in Chapters 10 and 11, do report some GC metrics, but these are aggregated data and the detail of individual GC events has been lost—and some of it can be very important for diagnosis.

I/O

File I/O has traditionally been one of the murkier aspects of overall system performance. Partly, this comes from its closer relationship with messy physical hardware, with engineers making quips about "spinning rust," but it is also because I/O lacks abstractions as clean as we see elsewhere in operating systems.

In the case of memory, the elegance of virtual memory as a separation mechanism works well. However, I/O has no comparable abstraction that provides suitable isolation for the application developer.

Fortunately, while most Java programs involve some simple I/O, the class of applications that heavily use the I/O subsystems is relatively small, and in particular, most applications do not simultaneously try to saturate I/O at the same time as either CPU or memory.

Not only that, but established operational practice has led to a culture in which production engineers are already aware of the limitations of I/O and actively monitor processes for heavy I/O usage.

For the performance analyst/engineer, it suffices to have an awareness of the I/O behavior of our applications. Tools such as `iostat` (and even `vmstat`) have the basic counters (e.g., blocks in or out), which are often all we need for basic diagnosis, especially if we make the assumption that only one I/O-intensive application is present per host.

Note that in virtualized environments (basically all cloud applications), I/O-intensive applications can give rise to what is known as the "noisy neighbor" problem—where one container has high requirements for things such as bandwidth or disk I/O and negatively affects the performance of other users running on the same underlying physical machine.

The performance engineer should pay particular attention to this possibility, as it may be difficult to detect directly.

Mechanical Sympathy

Mechanical sympathy is the idea that having an appreciation of the hardware is invaluable for those cases where we need to squeeze out extra performance.

> *You don't have to be an engineer to be a racing driver, but you do have to have mechanical sympathy.*
>
> —Jackie Stewart (attr)

The phrase was originally coined by Martin Thompson as a direct reference to Jackie Stewart and his car. However, as well as the extreme cases, it is also useful to have a baseline understanding of the concerns outlined in this chapter when dealing with production problems and looking at improving the overall performance of your application.

For many Java developers, mechanical sympathy is a concern that is possible to ignore. This is because the JVM provides a level of abstraction away from the hardware to unburden the developer from a wide range of

performance concerns. However, developers can use Java and the JVM quite successfully in the high-performance and low-latency space, by gaining an understanding of the JVM and the interaction it has with hardware. One important point to note is that the JVM actually makes reasoning about performance and mechanical sympathy harder, as there is more to consider.

Whether mechanical sympathy is important to your project or not will depend upon the application's business goals and service-level agreements.

Let's consider an example: the behavior of cache lines.

Earlier in this chapter, we discussed the benefit of processor caching. The use of cache lines enables the fetching of blocks of memory. In a multithreaded environment, cache lines can cause a problem when you have two threads attempting to read or write to a variable located on the same cache line, resulting in a race condition. The first thread will invalidate the cache line on the second thread, causing it to be reread from memory. Once the second thread has performed the operation, it will then invalidate the cache line in the first. This ping-pong behavior results in a drop-off in performance known as *false sharing*—but how can this be fixed?

Mechanical sympathy would suggest that first we need to understand that this is happening, and only after that, determine how to resolve it. In Java, the layout of fields in an object is not guaranteed, meaning it is easy to end up with variables sharing a cache line. One way to get around this would be to add padding around the variables to force them onto a different cache line.

Summary

Processor design and modern hardware have changed enormously. Driven by Moore's law and by engineering limitations (notably the relatively slow speed of memory), advances in processor design have become somewhat esoteric. The cache miss rate has become the most obvious leading indicator of how performant an application is.

In the Java space, the design of the JVM allows it to use additional processor cores even for single-threaded application code. This means that Java applications have received significant performance advantages from hardware trends, compared to other environments.

As Moore's law fades, attention will turn once again to the relative performance of software. Performance-minded engineers need to understand at least the basic points of modern hardware and operating systems to en-

sure that they can make the most of their hardware and not fight against it.

In the next chapter, we will move from the consideration of a single host and its hardware and operating system, and begin to consider the highly virtualized/containerized environments that increasingly represent the environments where Java applications are deployed.

[1] John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed. (Burlington, MA: Morgan Kaufmann, 2011).

[2] Access times shown in terms of number of clock cycles per operation; data provided by Google.

[3] According to documentation for the Intel Core i9-13900K Processor.

[4] Livio Soares and Michael Stumm, "FlexSC: Flexible System Call Scheduling with Exception-Less System Calls," in OSDI'10, *Proceedings of the 9th USENIX Conference on Operating Systems Design and Implementation* (Berkeley, CA: USENIX Association, 2010): 33–46.